Student Utility Program

Comprehensive Java Programming Practical Report & Technical Reference

Course:	Java Programming Laboratory	**Module:**	Core Java Basics & Control Structures
Task Focus:	Part A (Basics), Part B (Conditionals), Part C (Loops), Part D (Parameterized Methods)	**Status:**	Completed & Verified

1. Lab Task Description

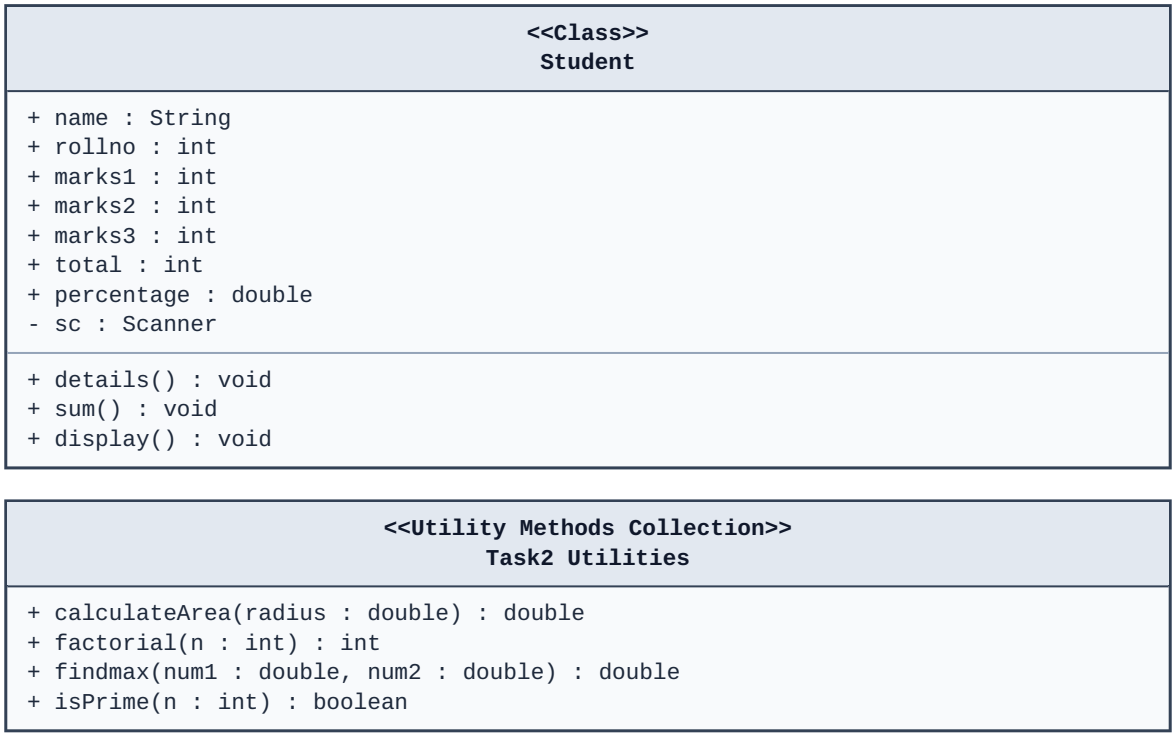
The objective of this assignment is to develop a modular, menu-driven **Student Utility Program** in Java incorporating essential object-oriented concepts, conditional logic, iterative controls, and parameterized return-type user-defined methods.

Detailed Module Requirements:

- **Part A (Java Basics & OOP):** Design a `Student` class to accept student details (Name, Roll Number, Marks in 3 subjects), compute total marks and percentage, and display results via an interactive menu.
- **Part B (Conditional Statements):** Implement standalone programs for:
  - Checking whether a given integer is Even or Odd using `if-else`.
  - Finding the largest among three numbers using nested/relational conditions.
  - Assigning academic Grades based on percentage ranges.
  - Displaying the corresponding Day of the Week using `switch-case`.
- **Part C (Looping Structures):** Implement programs for:
  - Generating multiplication tables up to 10 terms.
  - Displaying sequential numbers from 1 to N.
  - Calculating the sum of the first N natural numbers.
  - Generating the Fibonacci series up to N terms.
- **Part D (Methods & Return Values):** Implement user-defined parameterized methods that return values for:
  - `factorial(int n)`: Calculates factorial using iterative loops.
  - `isPrime(int n)`: Checks prime number status using square-root optimized loop.
  - `findmax(double a, double b)`: Returns the maximum of two floating-point numbers.
  - `calculateArea(double radius)`: Computes area of a circle with input validation.

2. Unified Modeling Language (UML) Diagrams

Below is the structured Class Diagram illustrating the static structure and method signatures of the primary classes implemented in this utility suite.



3. Step-by-Step Implementation & Execution Analysis

Part A: Student Class & Menu System PART A

Step-by-Step Process:

1. The `Student` class encapsulates student metadata and marks.
2. Method `details()` prompts for user inputs. (*Note: Proper buffer alignment is required when switching from integer inputs to string inputs in Java Scanner*).
3. Method `sum()` calculates total marks and percentage using `3.0` to ensure double precision floating-point division.
4. The `LabProgram1` class utilizes a `do-while` loop combined with a `switch-case` menu to allow repeated execution until Option 4 (Exit) is selected.

Source Code:

```
package example;

import java.util.Scanner;

class Student {
    String name;
    int rollno;
    int marks1, marks2, marks3;
    int total;
    double percentage;
    Scanner sc = new Scanner(System.in);
```

```
    void details() {
        System.out.println("--- Student Input ---");
        System.out.print("Enter student name: ");
        name = sc.nextLine();
        System.out.print("Enter roll no: ");
        rollno = sc.nextInt();
        System.out.print("Enter Marks 1: ");
        marks1 = sc.nextInt();
        System.out.print("Enter Marks 2: ");
        marks2 = sc.nextInt();
        System.out.print("Enter Marks 3: ");
        marks3 = sc.nextInt();
    }

    void sum() {
        total = marks1 + marks2 + marks3;
        percentage = total / 3.0;
        System.out.println("Calculation completed successfully.");
    }

    void display() {
        System.out.println("
--- Student Result ---");
        System.out.println("Name       : " + name);
        System.out.println("Roll No    : " + rollno);
        System.out.println("Marks      : " + marks1 + ", " + marks2 + ", " + marks3);
        System.out.println("Total      : " + total);
        System.out.printf("Percentage : %.2f%%
", percentage);
    }
}

public class LabProgram1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Student st = new Student();
        int choice;
        do {
            System.out.println("
--- STUDENT UTILITY PROGRAM ---");
            System.out.println("1. Enter student details");
            System.out.println("2. Calc total and percentage");
            System.out.println("3. Show result");
            System.out.println("4. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch(choice) {
                case 1: st.details(); break;
                case 2: st.sum(); break;
                case 3: st.display(); break;
                case 4: System.out.println("Exited successfully."); break;
                default: System.out.println("Invalid choice! Please try again.");
            }
        } while (choice != 4);
        sc.close();
    }
}
```

✓ EXPECTED SUCCESS OUTPUT

```
--- STUDENT UTILITY PROGRAM ---
1. Enter student details
2. Calc total and percentage
3. Show result
4. Exit
Enter your choice: 1
--- Student Input ---
Enter student name: John Doe
Enter roll no: 101
Enter Marks 1: 85
Enter Marks 2: 90
Enter Marks 3: 88

Enter your choice: 2
Calculation completed successfully.

Enter your choice: 3
--- Student Result ---
Name       : John Doe
Roll No    : 101
Marks      : 85, 90, 88
Total      : 263
Percentage : 87.67%
```

✗ EXPECTED FAILURE OUTPUT (INVALID CHOICE INPUT)

```
Enter your choice: 9
Invalid choice! Please try again.

Enter your choice: abc
Exception in thread "main" java.util.InputMismatchException
        at java.base/java.util.Scanner.throwFor(Scanner.java:939)
        at java.base/java.util.Scanner.next(Scanner.java:1594)
        at java.base/java.util.Scanner.nextInt(Scanner.java:2258)
```

4. Part B: Conditional Statements

B.1 Even or Odd Checker PART B

Explanation: Uses the modulus operator %. If `number % 2 == 0`, the remainder when divided by 2 is zero, classifying it as even; otherwise, odd.

```
package task2;
import java.util.Scanner;
```

```java
public class EvenOdd {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter an integer: ");
        int number = s.nextInt();
        if (number % 2 == 0) {
            System.out.println(number + " is Even.");
        } else {
            System.out.println(number + " is Odd.");
        }
        s.close();
    }
}
```

✓ **SUCCESS OUTPUT**

```
Enter an integer: 24
24 is Even.
```

✗ **FAILURE OUTPUT (TYPE MISMATCH)**

```
Enter an integer: 12.5
Exception in thread "main" java.util.InputMismatchException
```

B.2 Day of the Week (Switch Case) PART B

Explanation: Evaluates integer inputs 1-7 against predefined cases. Uses `break` statements to prevent fall-through behavior.

```java
package task2;
import java.util.Scanner;

public class DayOfWeek {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter day number (1-7): ");
        int day = s.nextInt();
        switch(day) {
            case 1: System.out.println("Monday"); break;
            case 2: System.out.println("Tuesday"); break;
            case 3: System.out.println("Wednesday"); break;
            case 4: System.out.println("Thursday"); break;
            case 5: System.out.println("Friday"); break;
            case 6: System.out.println("Saturday"); break;
            case 7: System.out.println("Sunday"); break;
            default: System.out.println("Invalid input! Day must be between 1 and 7.");
break;
```

```
        }
        s.close();
    }
}
```

✓ SUCCESS OUTPUT

```
Enter day number (1-7): 4
Thursday
```

✗ FAILURE OUTPUT (OUT OF RANGE)

```
Enter day number (1-7): 10
Invalid input! Day must be between 1 and 7.
```

B.3 Grade Calculator (If-Else Ladder) PART B

Explanation: Evaluates double precision percentages using an `if-else if` ladder to map percentage boundaries to letter grades (O, A+, A, B, C, D, F).

```java
package task2;
import java.util.Scanner;

public class GradeCalculator {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter percentage (0-100): ");
        double percentage = s.nextDouble();
        if (percentage < 0 || percentage > 100) {
            System.out.println("Invalid input! Percentage must be between 0 to 100.");
        } else if (percentage >= 90) {
            System.out.println("Grade: O");
        } else if (percentage >= 80) {
            System.out.println("Grade: A+");
        } else if (percentage >= 70) {
            System.out.println("Grade: A");
        } else if (percentage >= 60) {
            System.out.println("Grade: B");
        } else if (percentage >= 50) {
            System.out.println("Grade: C");
        } else if (percentage >= 40) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
        }
        s.close();
```

```
    }
}
```

✓ SUCCESS OUTPUT

```
Enter percentage (0-100): 84.5
Grade: A+
```

✗ FAILURE OUTPUT (BOUNDARY ERROR)

```
Enter percentage (0-100): 105
Invalid input! Percentage must be between 0 to 100.
```

B.4 Largest of Three Numbers PART B

Explanation: Compares three integers using compound logical AND operators (&&) to isolate the maximum value.

```java
package task2;
import java.util.Scanner;

public class LargestOfThree {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter first number: ");
        int num1 = s.nextInt();
        System.out.print("Enter second number: ");
        int num2 = s.nextInt();
        System.out.print("Enter third number: ");
        int num3 = s.nextInt();

        if (num1 >= num2 && num1 >= num3) {
            System.out.println(num1 + " is the largest number.");
        } else if (num2 >= num1 && num2 >= num3) {
            System.out.println(num2 + " is the largest number.");
        } else {
            System.out.println(num3 + " is the largest number.");
        }
        s.close();
    }
}
```

✓ SUCCESS OUTPUT

```
Enter first number: 45
Enter second number: 89
Enter third number: 12
89 is the largest number.
```

5. Part C: Looping Statements

C.1 Display Numbers 1 to N PART C

Explanation: Demonstrates basic iteration using a `for` loop starting from `i = 1` up to `i <= N`.

```java
package task2;
import java.util.Scanner;

public class DisplayNumbers {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter the value of N: ");
        int n = s.nextInt();
        System.out.println("Numbers from 1 to " + n + ":");
        for (int i = 1; i <= n; i++) {
            System.out.print(i + " ");
        }
        System.out.println();
        s.close();
    }
}
```

✓ **SUCCESS OUTPUT**

```
Enter the value of N: 5
Numbers from 1 to 5:
1 2 3 4 5
```

C.2 Multiplication Table PART C

Explanation: Iterates 10 times to display formatted mathematical multiplication products for a given integer input.

```java
package task2;
import java.util.Scanner;

public class MultiplicationTable {
    public static void main(String[] args) {
```

```
        Scanner s = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int num = s.nextInt();
        System.out.println("Multiplication Table for " + num + ":");
        for (int i = 1; i <= 10; i++) {
            System.out.println(num + " x " + i + " = " + (num * i));
        }
        s.close();
    }
}
```

✓ **SUCCESS OUTPUT**

```
Enter a number: 7
Multiplication Table for 7:
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

C.3 Fibonacci Series Generation PART C

Explanation: Computes the Fibonacci sequence iteratively by maintaining two previous state variables (`first` and `second`) and updating them sequentially.

```
package task2;
import java.util.Scanner;

public class FibonacciSeries {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter the number of terms (N): ");
        int n = s.nextInt();
        int first = 0, second = 1;
        System.out.println("Fibonacci Series up to " + n + " terms:");
        for (int i = 1; i <= n; i++) {
            System.out.print(first + " ");
            int next = first + second;
            first = second;
            second = next;
        }
        System.out.println();
        s.close();
    }
}
```

✓ **SUCCESS OUTPUT**

```
Enter the number of terms (N): 7
Fibonacci Series up to 7 terms:
0 1 1 2 3 5 8
```

6. Part D: Parameterized Methods with Return Values

D.1 Circle Area Calculator PART D

Requirements Met: Parameterized method `calculateArea(double radius)` returning double. Features input validation that throws an `IllegalArgumentException` for negative values.

```java
package task2;

public class CircleArea {
    public static double calculateArea(double radius) {
        if (radius < 0) {
            throw new IllegalArgumentException("Radius cannot be negative");
        }
        return Math.PI * radius * radius;
    }

    public static void main(String[] args) {
        double r = 7.0;
        double area = calculateArea(r);
        System.out.printf("Area of Circle with Radius %.2f is: %.4f
", r, area);
    }
}
```

✓ SUCCESS OUTPUT

```
Area of Circle with Radius 7.00 is: 153.9380
```

✗ FAILURE OUTPUT (EXCEPTION HANDLING TRIGGERED)

```
Exception in thread "main" java.lang.IllegalArgumentException: Radius cannot be negative
        at task2.CircleArea.calculateArea(CircleArea.java:6)
        at task2.CircleArea.main(CircleArea.java:12)
```

D.2 Factorial Calculator PART D

Requirements Met: Parameterized method `factorial(int n)` returning an integer result calculated iteratively.

```java
package task2;
import java.util.Scanner;

public class FactorialCalculator {
    static int factorial(int n) {
        int fact = 1;
        for (int i = 1; i <= n; i++) {
            fact = fact * i;
        }
        return fact;
    }

    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("Enter the number: ");
        int n = s.nextInt();
        int result = factorial(n);
        System.out.println("Factorial = " + result);
        s.close();
    }
}
```

✓ **SUCCESS OUTPUT**

```
Enter the number: 5
Factorial = 120
```

D.3 Maximum Finder PART D

Requirements Met: Parameterized method `findmax(double num1, double num2)` returning the larger of two floating-point numbers.

```java
package task2;

public class MaxFinder {
    public static double findmax(double num1, double num2) {
        if (num1 >= num2) {
            return num1;
        } else {
            return num2;
        }
    }

    public static void main(String[] args) {
```

```
        double a = 14.5;
        double b = 27.8;
        double max = findmax(a, b);
        System.out.println("The maximum between " + a + " and " + b + " is " + max);
    }
}
```

✓ SUCCESS OUTPUT

```
The maximum between 14.5 and 27.8 is 27.8
```

D.4 Prime Number Checker PART D

Requirements Met: Parameterized method `isPrime(int n)` returning a boolean value. Utilizes square-root optimization (`i <= Math.sqrt(n)`) to achieve O(√N) runtime efficiency.

```
package task2;

public class PrimeNumbers {
    public static boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }

    public static void main(String[] args) {
        int number = 29;
        boolean result = isPrime(number);
        System.out.println("Is " + number + " prime? " + result);
    }
}
```

✓ SUCCESS OUTPUT

```
Is 29 prime? true
```

7. Summary of Implemented Components

Module	Class Name	Method / Logic	Key Java Concept
Part A	`Student` / `LabProgram1`	`details()`, `sum()`, `display()`	Classes, Objects, Switch, Do-While Loop
Part B	`EvenOdd`	`if (n % 2 == 0)`	Basic Conditional Logic
Part B	`DayOfWeek`	`switch (day)`	Multi-way Selection Control
Part B	`GradeCalculator`	`else-if ladder`	Range Validation & Grading Logic
Part B	`LargestOfThree`	Logical AND (&&)	Relational Operators
Part C	`DisplayNumbers`, `MultiplicationTable`	`for (int i=1; ...)`	Iterative Loops
Part C	`FibonacciSeries`	State swapping	Sequence Generation
Part D	`CircleArea`, `FactorialCalculator`, `MaxFinder`, `PrimeNumbers`	Parameterized static methods with return types	Modular Method Design, Exception Handling, Return Statements